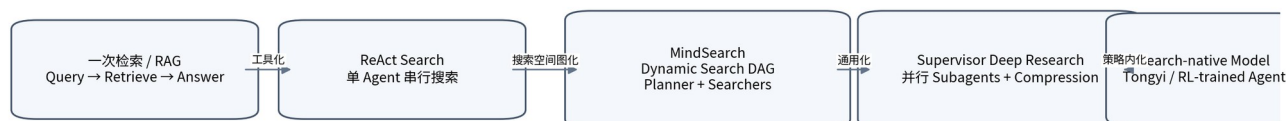


# MindSearch

## 完整研究：Dynamic Web Search Graph 与多 Agent 深度搜索

论文机制、源码架构、实验消融、Context Engineering 与可复用 Know-why / Know-how

从 RAG 到 Search Graph：MindSearch 在 Deep Research 演化中的位置



研究基线：MindSearch ICLR 2025 论文 / arXiv 2407.20183、InternLM/MindSearch 官方仓库与 Release、OpenReview / ICLR Proceedings。研究日期：2026-08-13。

### 核心判断

MindSearch 的本质不是“搜索 Agent 并行化”，而是把复杂信息需求转化为一个由 LLM 动态构造、可执行、可验证、可继续扩展的搜索 DAG；WebSearcher 再把海量网页压成局部证据，使 Planner 不必直接吞下整个 Web Context。

# 目录与执行摘要

- 1. 时代背景：为什么 ReAct Search 不够
- 2. 总体架构：WebPlanner + WebSearcher
- 3. Dynamic Web Search Graph
- 4. Code-as-Planning 与 DAG 执行协议
- 5. WebSearcher 的 Hierarchical Retrieval
- 6. Multi-Agent = Context Scaling Primitive
- 7. 搜索循环、并发与停止
- 8. Open-set Human Evaluation
- 9. Closed-set QA 实验
- 10. Planner / Searcher 消融
- 11. 时间效率、DAG 深度与错误恢复
- 12. 官方仓库与工程实现
- 13. MindSearch vs STORM
- 14. MindSearch vs GPT Researcher / Open Deep Research
- 15. MindSearch vs Tongyi / DeerFlow
- 16. 局限：事实性、引用与长上下文
- 17. 可复用 Know-why / Know-how
- 18. 如何实现一个现代 MindSearch-style Research Agent
- 19. 下一代改进方向
- 20. 来源与证据等级

## 一句话

如果 STORM 的创新是“先把问题问对，再写”，那么 MindSearch 的创新是“把问题分解与搜索依赖关系变成一个运行时不断生长的图”。

## MindSearch 总体架构：WebPlanner 动态建图 + WebSearcher 层级检索

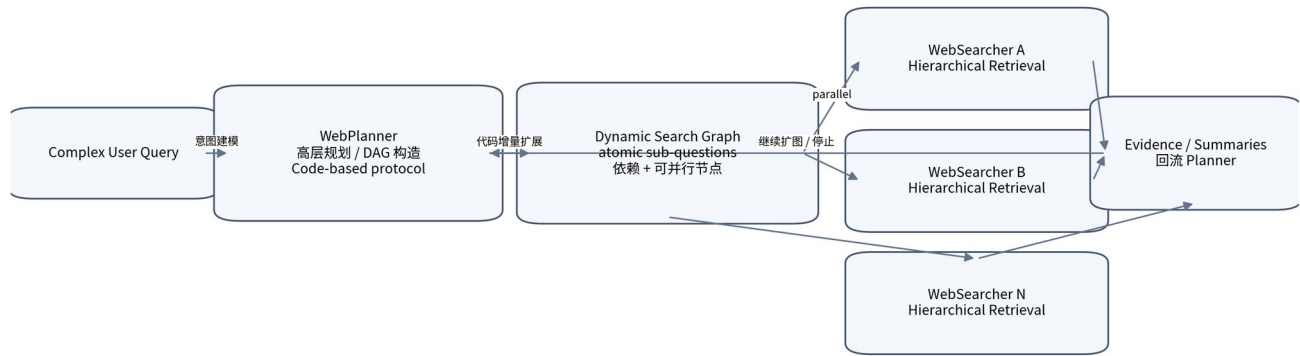


图1 MindSearch 的两层核心：Planner 负责搜索空间，Searcher 负责证据空间

# 1. 时代背景：为什么 ReAct Search 不够

MindSearch 的论文从三个非常工程化的问题出发：复杂请求无法通过一次检索完整命中；有用信息分散在大量网页并混有大量噪声；迭代搜索产生的网页正文很容易超过 LLM 的上下文容量。

这三个问题分别对应三个系统层面的矛盾：**\*\*Query Space 不够结构化、Evidence Space 太大、Context Window 太小\*\***。普通“LLM + Search Tool”通常只解决“模型可以搜索”，没有解决“应该搜索什么、哪些搜索可并行、何时追问、哪些网页值得深读、哪些结果应该回到主上下文”。

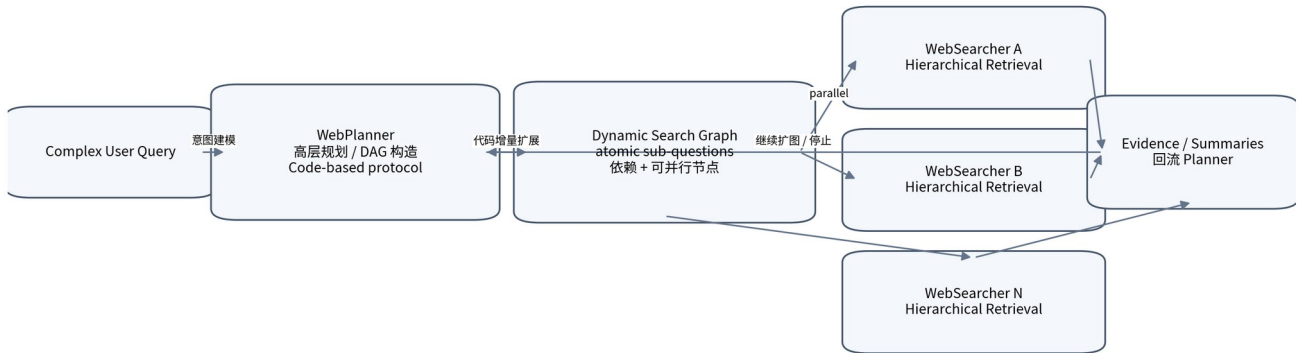
| 问题   | 朴素方案                         | 失败原因                    | MindSearch 对策                                          |
|------|------------------------------|-------------------------|--------------------------------------------------------|
| 复杂意图 | 直接把原问题交给搜索引擎                 | 搜索引擎更擅长原子查询，不擅长一次覆盖多跳意图 | WebPlanner 拆成原子子问题并建 DAG                               |
| 网页噪声 | 把 top-k snippet / 页面全部塞给 LLM | 海量无关内容稀释有效证据            | WebSearcher 做 multi-query + page selection + deep read |
| 长上下文 | 单 Agent 保存所有搜索历史             | 随着搜索轮数增长迅速爆 Context     | Searcher 独立上下文，只回传摘要                                   |
| 规划错误 | 自然语言 ReAct 动作                | 动作格式弱、难验证、依赖关系不显式       | 代码式规划 + DAG 状态 + 执行反馈                                  |

## Know-why

Deep Research 的第一性问题并不是“接更多搜索 API”，而是把 Search Policy、Evidence Selection、Context Budget 三件事同时设计。

## 2. 总体架构：WebPlanner + WebSearcher

MindSearch 总体架构：WebPlanner 动态建图 + WebSearcher 层级检索



论文把系统刻意压缩成两个主要 Agent。WebPlanner 是高层控制面：负责理解复杂问题、生成子问题、维护搜索依赖、决定并行分支、根据结果继续扩图并最终综合。WebSearcher 是数据面：接受一个具体子问题，完成查询改写、搜索、页面筛选、深读与局部总结。

这种二层设计与“多个角色 Agent”不同。角色名称不是重点，真正的隔离边界是 \*\*规划状态\*\* 与 \*\*原始网页上下文\*\*：Planner 只应该看足以做下一步决策的信息；Searcher 才承担重网页 I/O 与长文本读取。

### 工程抽象

Control Plane = WebPlanner；Data Plane = WebSearcher。复杂系统中，把“决定下一步做什么”和“处理大量数据”分开，往往比给 Agent 增加更多人格/角色更有效。

### 3. Dynamic Web Search Graph：搜索空间为什么要图化

Dynamic DAG 的意义：不是一次性 Plan，而是“搜索结果驱动的增量规划”



MindSearch 将复杂问题的搜索过程建模为动态图。节点表示原子子问题/搜索任务，边表示依赖关系。关键点在“dynamic”：图不是在研究开始前一次性生成完，而是根据 WebSearcher 返回的新证据继续扩展。

因此它和传统 Planner 的差别是：传统 Plan 是静态任务清单；MindSearch 的 DAG 是一个 \*\*可执行的 Research State\*\*。某个节点完成后，Planner 可以发现新的信息缺口、新实体或新依赖，进而生成后续节点。互不依赖的节点则能并行执行。

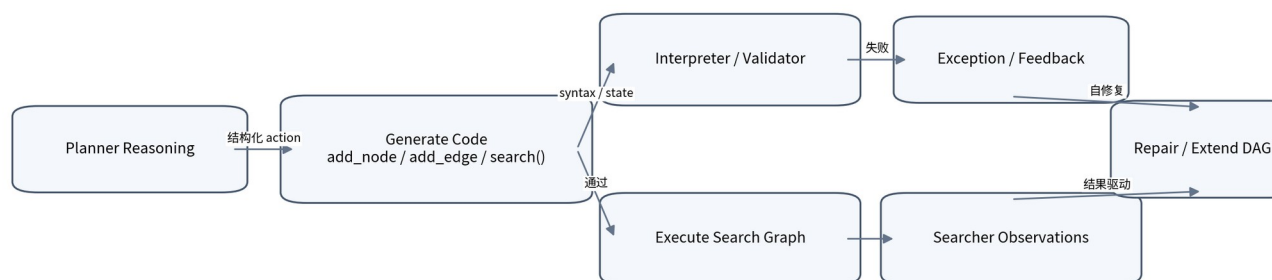
- Graph node = 一个足够原子的 research unit，目标应该可独立检索并给出局部结论。
- Graph edge = 信息依赖，不只是“步骤顺序”。有依赖时串行，没有依赖时并行。
- Graph expansion = Evidence-driven planning。新的搜索动作来自当前证据，而不是只来自初始问题。
- Graph termination = 当核心问题覆盖充分、边际搜索收益降低，或达到预算后结束。

#### Know-how

把 Research Plan 从“文本计划”升级为“显式状态图”后，才能可靠做并发、重试、缓存、节点级预算、可观测性和错误恢复。

## 4. Code-as-Planning：为什么 Planner 用代码而不是纯 ReAct

Code-as-Planning：代码不仅是动作格式，也是规划校验与错误恢复接口



论文的 Planner 消融显示：相同 WebSearcher 下，无搜索 37.6、ReAct 58.0、CodeAct 61.3、MindSearch 64.0。作者认为代码提供统一动作空间，并且能通过解释器验证语法与执行状态。

这里的核心并不是“代码比自然语言更聪明”，而是代码天然具有 \*\*结构、引用、依赖、可执行性、异常反馈\*\* 。当 Planner 产生非法代码或错误规划语句时，运行时可以把异常重新反馈给模型，让它修复。

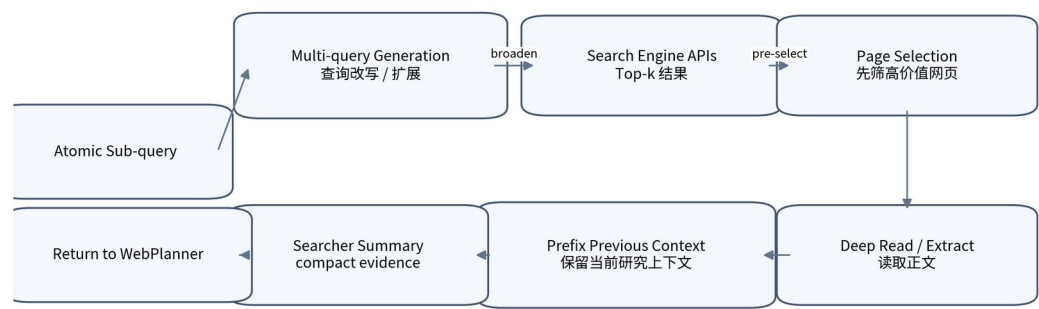
这是一条非常可迁移的 Agent 原则：当任务需要维护复杂中间状态时，不要只要求模型输出自然语言计划；让模型输出一种 host 可以解析、验证和执行的中间表示（IR）。IR 可以是 Python、JSON DSL、Graph commands 或 typed tool calls。

### 可迁移原则

LLM 负责生成候选策略；Runtime 负责校验状态转移。可验证的 action language 会显著降低长链路 Agent 的脆弱性。

# 5. WebSearcher：Hierarchical Retrieval 的完整逻辑

WebSearcher：分层检索把“海量网页”压成 Planner 可消费的证据



WebSearcher 不是“搜索一次然后摘要”。论文和项目材料描述的是分层检索：先做多查询生成以扩大覆盖，再调用搜索引擎获取候选页面；随后进行 Page Selection，只选择高价值网页进行更深阅读，最后提取信息并形成局部总结。

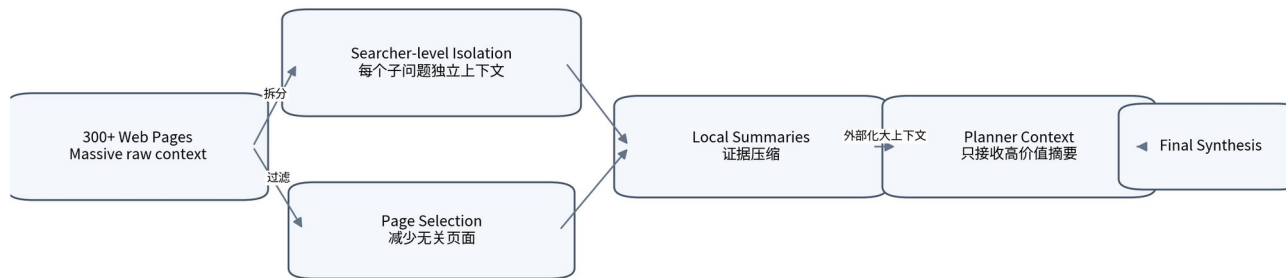
论文消融非常有价值：完整方法 64.0；去掉 Multi-Query Generation 为 60.6；去掉 Prefix Previous Context 为 63.3；去掉 Page Selection 直接降到 58.0。Page Selection 的影响最大，说明 Deep Research 很容易误以为“更多网页 = 更好”，实际上检索后过滤是核心能力。

| 组件             | 目的                | 去掉后的结果      | 工程启示                         |
|----------------|-------------------|-------------|------------------------------|
| MQG            | 扩大 query coverage | 64.0 → 60.6 | 复杂问题需要 query diversification |
| PPC            | 让当前搜索保留已有局部上下文    | 64.0 → 63.3 | 上下文延续有帮助，但不是最大增益             |
| Page Selection | 在深读前筛选候选网页        | 64.0 → 58.0 | 网页预算应花在“最值得读”的页面             |

**Know-why**  
Search API 解决的是 Recall；Page Selection 解决的是 Precision；Deep Read / Extraction 解决的是 Evidence Density。三者不能混为一个“search tool”。

## 6. Multi-Agent 的真实价值：Context Scaling Primitive

Context Engineering：多 Agent 不是角色扮演，而是上下文分片与压缩机制



论文强调其多 Agent 设计可以处理超过 300 个网页，并把大量信息分派给不同 WebSearcher。这里最值得学习的不是“多 Agent 比单 Agent 更聪明”，而是 \*\*多 Agent 把原始上下文分片\*\*。

如果 Planner 亲自读取每一个网页，主上下文会同时混入：问题分解、搜索日志、网页正文、页面摘要、工具错误和中间答案。MindSearch 将网页级数据留在 Searcher，Planner 只接收压缩结果，从而把 Context Window 从“数据总线”恢复为“决策工作区”。

这与后来 Open Deep Research 的 subagent isolation + compression、DeerFlow 的 filesystem externalization、Claude Code 类 runtime 的 context engineering 本质一致：\*\*信息的物理存放位置和推理上下文不是一回事\*\*。

### Know-how

设计 Subagent 时先问“我要隔离哪一类 Context”，再问“这个 Agent 叫什么角色”。上下文边界比角色命名重要。

## 7. 搜索循环、并发与停止：从 Tree-of-Thought 到 Runtime Graph

MindSearch 的运行可以抽象为：Planner 读取当前 Graph State → 选择可执行节点 → 并行调用 Searchers → 写回局部证据 → 判断 coverage gap → 新增节点/边 → 重复。

```
while not done:
    ready = graph.ready_nodes()
    results = await parallel(web_searcher(node) for node in ready)
    graph.attach(results)
    new_nodes, new_edges = planner.expand(graph.summary())
    graph.update(new_nodes, new_edges)
    done = planner.should_stop(graph, budget)
```

生产实现必须额外补齐论文原型没有充分强调的预算：max graph nodes、max graph depth、per-searcher timeout、total web pages、total tokens、global wall-clock、duplicate-query detection、domain diversity、failure policy。否则动态建图很容易退化成失控扩张。

### 工程重点

DAG 给你并行能力，也给你预算治理的天然粒度：预算可以绑定到 node / edge / depth / branch，而不是只有一个粗糙的全局 max\_steps。

## 8. Open-set Human Evaluation：MindSearch 强在哪里

论文针对开放式 QA，让人类专家比较 MindSearch、ChatGPT-Web 与 Perplexity.ai Pro，评价维度包括 Depth、Breadth 和 Factuality。论文报告 MindSearch 在深度和广度上获得显著偏好；作者也明确承认事实性提升没有广度提升那么明显，并把这归因于更细搜索可能增加长上下文干扰。

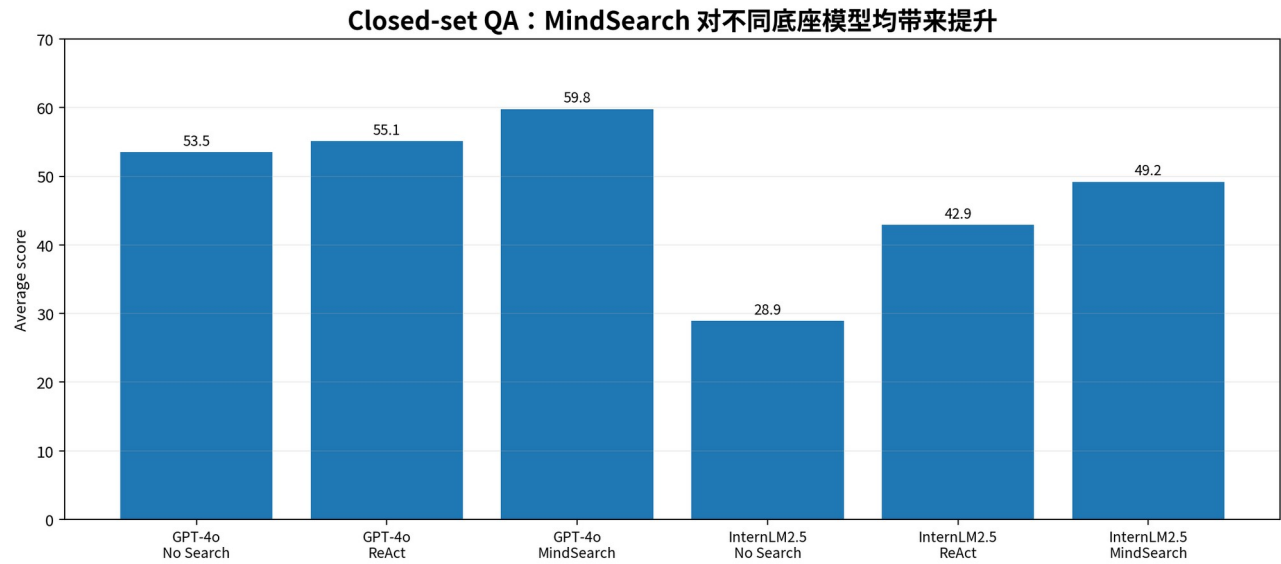
这个结果很重要：Deep Research 的“更深、更广”与“更真实”不是同一个目标。增加搜索分支可以提高覆盖，但如果 Citation/Provenance/Contradiction Handling 没同步增强，系统可能得到更多细节，同时累积更多噪声或错误。

| 评价维度       | MindSearch 的主要收益 | 潜在代价               |
|------------|------------------|--------------------|
| Depth      | 更细粒度、多跳搜索路径      | 链路更长，错误可传播         |
| Breadth    | 并行 DAG 覆盖多个子主题   | 网页量增大，证据冲突增多       |
| Factuality | 搜索提供外部知识支撑       | 并不会随搜索量线性提高，需要证据治理 |

### 关键结论

Research Quality  $\neq$  Search Volume。一个成熟系统必须分别优化 Coverage、Evidence Quality、Citation Accuracy、Synthesis Faithfulness。

## 9. Closed-set QA：实验结果与真正说明的问题



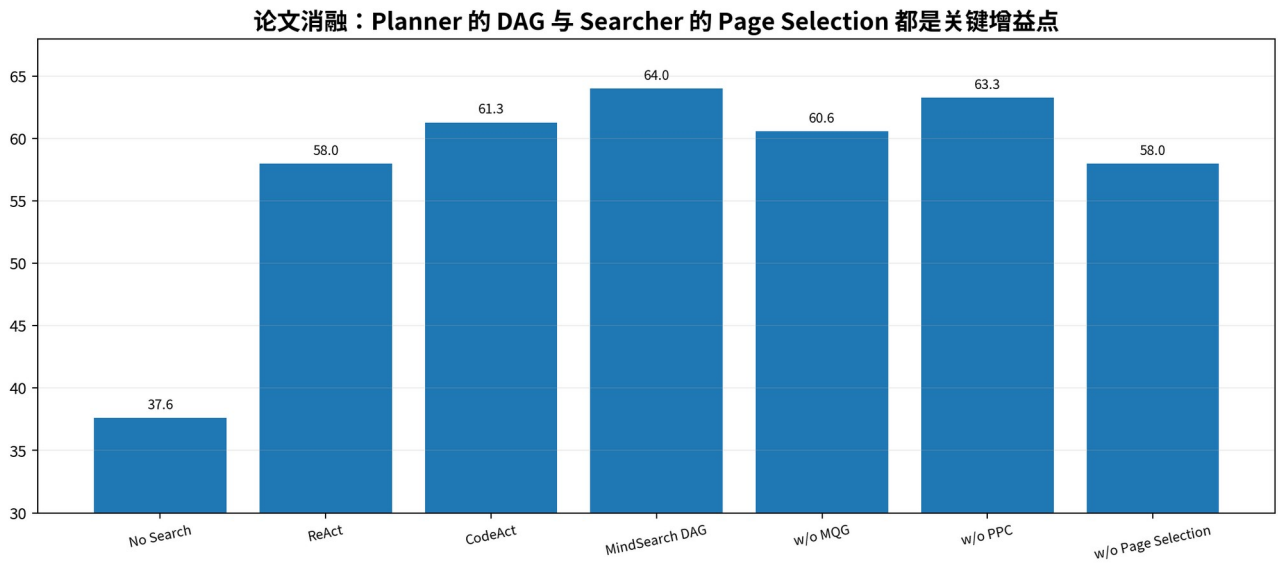
| Backend        | Method       | Bamboogle | Musique 2/3/4-hop  | HotpotQA E/M/H | AVG  |
|----------------|--------------|-----------|--------------------|----------------|------|
| GPT-4o         | No Search    | 70.4      | 54.0 / 22.0 / 20.0 | 73 / 69 / 66   | 53.5 |
| GPT-4o         | ReAct Search | 75.2      | 48.0 / 25.0 / 13.3 | 81 / 73 / 70   | 55.1 |
| GPT-4o         | MindSearch   | 76.8      | 60.0 / 35.0 / 14.6 | 80 / 74 / 78   | 59.8 |
| InternLM2.5-7B | No Search    | 34.0      | 28.0 / 10.0 / 17.3 | 47 / 26 / 40   | 28.9 |
| InternLM2.5-7B | ReAct Search | 55.2      | 38.0 / 17.0 / 16.0 | 69 / 56 / 49   | 42.9 |
| InternLM2.5-7B | MindSearch   | 67.8      | 46.0 / 20.0 / 18.6 | 69 / 66 / 57   | 49.2 |

论文最有启发的不是绝对分数，而是“小模型增益更大”：InternLM2.5-7B 从无搜索 28.9 → ReAct 42.9 → MindSearch 49.2。说明好的 Search Runtime 可以显著补偿基础模型的知识与规划短板。

### Know-why

Agent 系统性能是 Model × Runtime × Tools × Context 的乘积。不能只用“换更强模型”解释所有提升。

# 10. 消融实验：哪些设计是真的有效



Planner 消融：No Search 37.6、ReAct 58.0、CodeAct 61.3、MindSearch 64.0。Searcher 消融：完整 64.0、w/o MQG 60.6、w/o PPC 63.3、w/o Page Selection 58.0。

可归纳为两个核心结论：第一，\*\*规划表示\*\*会影响搜索质量，显式代码/DAG 优于纯自然语言串行 ReAct；第二，\*\*证据预选择\*\*比“把更多页面交给模型”更重要。

这也解释了为什么很多 Deep Research Demo 看起来会搜索很多，却不一定得到好报告：它们往往把“搜索次数”当作研究深度，把“抓取全文”当作证据质量，而缺少 Planner State 和 Page Selection。

# 11. 时间效率、DAG 深度与错误恢复

论文附录给出一个非常直观的时间实验：10 个复杂研究问题分配给 5 位人类专家，共耗时约 19 小时 17 分；MindSearch 完成相同任务约 23 分钟。单个人类样例中，搜索/阅读 100+ 网页约 47 分钟，写约 3,000 字回答再花 74 分钟。这个实验主要说明并行检索与自动整理的吞吐优势，而不是证明系统在所有质量维度都等价于人类。

论文还在 Musique 上观察问题 hop 数与生成 DAG 深度：2-hop → 1.1，3-hop → 1.2，4-hop → 1.6。深度没有等于 hop 数，因为独立问题可并行执行，也可能存在 shortcut。

| Musique hops | 平均 DAG depth | 解释                 |
|--------------|--------------|--------------------|
| 2            | 1.1          | 多个依赖可在同一层并行        |
| 3            | 1.2          | 不是每个逻辑 hop 都需要一层搜索 |
| 4            | 1.6          | 更复杂问题仍会促使图变深       |

另外，代码执行协议提供了错误恢复通道：无效代码/规划可由解释器报错，再把异常反馈给 Planner 重新生成。这是 Runtime Feedback 驱动 self-repair 的早期典型。

## 12. 官方仓库与工程实现现状（截至 2026-08-13）

官方仓库 InternLM/MindSearch 当前页面显示约 6.9k Stars、约 690 Forks、56 commits。README 的最新显著 changelog 是 2024-11-05：Agent 模块基于 Lagent v0.5 重构，以改善并发性能，并更新 UI 展示 simultaneous multi-query search。2026 年仍有 issue / PR 活动，但项目整体迭代频率明显低于一些仍高速演进的 Deep Research harness。

| 工程面             | 官方实现/说明                                             |
|-----------------|-----------------------------------------------------|
| Backend         | FastAPI；可运行异步 Agent (--asy)                         |
| Agent framework | Lagent；2024-11 重构到 v0.5                             |
| Model           | README 示例支持 InternLM2.5-7B server 与 GPT-4，可改模型配置    |
| Search          | DuckDuckGo / Bing / Brave / Google Serper / Tencent |
| Frontend        | React / Gradio / Streamlit                          |
| License         | Apache-2.0                                          |
| Debug           | python -m mindsearch.terminal；backend_example.py    |

### 工程评价

MindSearch 更像“研究原型 + 可运行 AI Search Engine”，而不是 2026 年意义上的通用 Agent Harness。它最值得复用的是算法结构，不是直接照搬整套 runtime。

### 13. MindSearch vs STORM：两个 2024 年重要方向的分叉

STORM 和 MindSearch 都认为“复杂问题不能直接一次搜索”。但两者选了不同的结构化对象：STORM 结构化的是 \*\*问题视角与最终知识结构（Outline）\*\*；MindSearch 结构化的是 \*\*搜索依赖与执行轨迹（DAG）\*\*。

| 维度   | STORM                                            | MindSearch                    |
|------|--------------------------------------------------|-------------------------------|
| 核心对象 | Perspectives + simulated conversations + outline | Dynamic search DAG            |
| 问题发现 | 从相似主题视角产生多角度问题                                   | 从当前搜索状态动态扩展子问题                |
| 执行结构 | 多视角访谈，之后统一写作                                     | 依赖感知的并行 Searcher              |
| 知识组织 | Outline + Reference Pool                         | Graph State + local summaries |
| 强项   | 长报告结构、主题覆盖                                       | 动态多跳、并行搜索路径                   |
| 弱项   | 实时动态纠偏相对弱                                        | 最终结构与证据治理相对弱                  |

**组合思路**

最佳现代系统不必二选一：可以用 MindSearch DAG 做 Research Execution，用 STORM-style Outline / Concept Map 做 Knowledge Organization。

# 14. 与 GPT Researcher / Open Deep Research 的差异

MindSearch 与 Deep Research 主流范式对比

| 系统                  | 规划结构                    | 检索执行                       | Context 控制                       | 核心优势       | 主要短板       |
|---------------------|-------------------------|----------------------------|----------------------------------|------------|------------|
| STORM               | 多视角问题 + Outline         | 模拟访谈/检索                    | Reference Pool + Section Context | 覆盖广、结构好    | 动态追问能力较弱   |
| MindSearch          | 动态 DAG / Code           | 多 WebSearcher 并行           | Searcher 隔离 + 摘要                 | 搜索路径可动态扩展  | 事实性与证据治理较弱 |
| GPT Researcher      | 子查询 / 递归                | Search + Scrape + Compress | Context Compressor               | 工程生态成熟     | 规划拓扑较传统    |
| Open Deep Research  | Supervisor tool-calling | Subagent ReAct             | Subagent 隔离 + Compression        | 通用、模型驱动    | 依赖强模型工具能力  |
| Tongyi DeepResearch | 策略内化到模型                 | Native ReAct Search        | 训练学得长程策略                         | Agent 能力内化 | 训练/环境成本高   |
| DeerFlow 2.0        | Harness + Skills        | 动态工具/子代理                   | Filesystem + Summary + Memory    | 通用长时程运行时   | 系统复杂度高     |

GPT Researcher 更像工程化 Research Pipeline：query planning → search → scrape → context compression → report；Open Deep Research 更像 Supervisor + Subagent 的通用 tool-calling research agent。MindSearch 更早、更明确地把“搜索计划”建模为 Graph Runtime。

Open Deep Research 的优势是模型可以直接决定何时调用 ConductResearch、何时 think、何时结束，减少手写 graph semantics；MindSearch 的优势是依赖结构更显式、并发与节点状态更自然。两者体现了一个长期张力：\*\*显式 Workflow / Graph Control vs 模型驱动的 Emergent Planning\*\*。

可迁移判断

如果任务依赖关系强、需要缓存/复用/可视化/审计，显式 DAG 很有价值；如果任务开放性高、模型工具能力强，Supervisor loop 往往更灵活。

## 15. MindSearch vs Tongyi DeepResearch / DeerFlow 2.0

Tongyi DeepResearch 把搜索策略进一步“内化”进模型：通过 Agentic Continual Pretraining、SFT 与 RL，让模型直接学习长程搜索和工具交互；MindSearch 则把策略主要放在外部 Agent Runtime 与 Prompt / Code protocol 中。

DeerFlow 2.0 则走向另一个方向：Research 不再是固定主架构，而成为一个 Skill，运行时提供通用 Lead Agent、middleware、sandbox、subagents、memory、filesystem 与 guardrails。MindSearch 的 Dynamic Graph 可以看成是一个强研究策略模块，但并不等同于完整长时程 Agent Runtime。

| 系统                  | Research Policy 在哪里                               | Runtime 通用性 | 训练成本 |
|---------------------|---------------------------------------------------|-------------|------|
| MindSearch          | Planner prompt + DAG protocol + Searcher pipeline | 中           | 低    |
| Tongyi DeepResearch | 模型权重 / RL policy                                  | 低到中（研究专用）   | 高    |
| DeerFlow 2.0        | Skill + Lead Agent runtime                        | 高           | 低到中  |

# 16. 局限：事实性、引用、长上下文与生产可靠性

论文自己指出 MindSearch 的事实性提升不如深度/广度明显。其原因不仅是“模型长上下文能力不足”，还包括现代 Deep Research 更关注的几个维度：source authority、citation correctness、claim-to-source entailment、contradiction resolution、freshness、duplicate evidence、source diversity。

- 事实性：搜索分支越多，不代表最终综合越可靠；错误会在多跳总结中累计。
- 引用：论文主要关注答案质量与搜索能力，现代系统需要 claim-level provenance 和 citation verification。
- 长上下文：局部摘要能减少 token，但压缩过程本身可能丢失限定条件、数字或冲突信息。
- 动态图膨胀：没有预算/重复检测/边际收益判断时，Graph expansion 容易失控。
- 网页安全与质量：需要域名 allow/deny、恶意页面防护、prompt injection 防御、内容可信度分级。
- 工程成熟度：官方仓库主要是研究/演示框架，生产环境还需要持久状态、任务恢复、监控、成本治理与测试。

## 升级方向

从“Search Graph”升级为“Evidence Graph”：节点不仅记录问题和答案，还记录 claim、source、confidence、time、contradiction 与 citation links。

## 17. 可累积的 Know-why / Know-how

### 1. 把搜索空间显式化

复杂 research 不是一个 query list，而是有依赖的状态空间。显式 graph 能承载并行、重试、预算和可观测性。

### 2. 计划必须是动态的

一次性 Plan 无法利用新证据；每轮结果都应该能改变后续 query policy。

### 3. Search 与 Read 分离

搜索 API 返回候选，不等于证据。必须单独设计 Page Selection / Deep Read / Extraction。

### 4. Subagent 的核心是 Context Isolation

多 Agent 的首要价值是把大体积网页数据留在局部 context，而不是“集体智慧”的叙事。

### 5. 动作语言应可验证

Code / typed tools / DSL 让 Host 能检查状态转移，并把异常反馈给模型自修复。

### 6. 并行必须基于依赖

真正高效的并发来自 DAG ready-set，不是盲目把所有子问题一起跑。

### 7. Coverage 与 Factuality 分开优化

多搜解决 breadth，不自动解决 citation accuracy 和 synthesis faithfulness。

### 8. 弱模型可以被强 runtime 放大

实验表明 InternLM2.5-7B 的相对增益明显，说明 Agent architecture 是独立能力来源。

# 18. 如何实现一个现代 MindSearch-style Research Agent

推荐不要复制 2024 代码，而是保留核心思想，使用现代 typed state + async runtime 重写。最小架构：

```
ResearchState:
  question
  graph: nodes, edges, status
  evidence_store
  budgets

Planner:
  expand(state_summary) -> GraphDelta

Searcher(node):
  generate_queries -> search -> dedupe -> rank_pages
  -> fetch -> extract_claims -> local_summary

Executor:
  ready_nodes = graph.topological_ready()
  run ready_nodes with bounded concurrency
  persist results / retry failures

Writer:
  synthesize from evidence graph with verified citations
```

| 阶段        | 建议实现                               | 必须有的 Guardrail                          |
|-----------|------------------------------------|-----------------------------------------|
| Planner   | structured output GraphDelta       | max nodes/depth、duplicate query         |
| Search    | multi-query + provider abstraction | per-domain limits、timeout               |
| Page Rank | LLM/embedding/reranker             | source authority / recency              |
| Fetch     | browser/fetch service              | prompt injection sanitizer              |
| Evidence  | claim extraction + provenance      | source URL / timestamp / quote span     |
| Executor  | async DAG scheduler                | global concurrency / budget             |
| Writer    | outline + claim-grounded synthesis | citation verifier / contradiction check |

MVP 顺序

先做 DAG State + Searcher + bounded async executor；再加 page selection；再加 evidence store；最后才加复杂 UI 和多模型路由。

## 19. 下一代 MindSearch：从 Search DAG 到 Evidence-Centric Research Runtime

如果在 2026 年重做 MindSearch，最值得升级的是把动态 DAG 与现代 Context Engineering / Evidence Verification 结合。

- Graph Node 不只存 “sub-query”，还存 expected evidence、completion criteria、priority、estimated cost。
- 增加 Evidence Store：claim → source → excerpt → timestamp → confidence → supporting/contradicting edges。
- 用 marginal information gain / uncertainty reduction 决定是否扩图，而不是仅靠模型直觉。
- 对每个节点做 source diversity 与 domain authority 约束，减少同质网页复读。
- 采用 filesystem/object store 外部化原始页面，只把 summaries + evidence handles 放进 LLM context。
- 加入 Loop Detection、budget middleware、retry policy、checkpoint/resume、run event journal。
- 最终报告生成前运行 Citation Verifier：每个关键 claim 至少有一条可验证 source，冲突 claim 必须显式说明。
- 把 STORM 的 Outline-first 或 Co-STORM 的 Mind Map 作为最终知识组织层，弥补 MindSearch 对 report structure 的弱化。

### 统一公式

Deep Research  $\approx$  Dynamic Planning  $\times$  Hierarchical Retrieval  $\times$  Context Isolation  $\times$  Evidence Selection  $\times$  Verified Synthesis  $\times$  Runtime Budgeting。

## 20. 来源、证据等级与阅读顺序

以下优先使用官方/一手来源。论文中的数值与方法说明以 ICLR / OpenReview / arXiv 为准；仓库状态以 GitHub 官方页面为准；alphaXiv 仅用于快速定位论文表格和附录文本，重要数值与论文版本交叉核对。

| ID | 来源                    | 用途                                                                     | URL                                                                                                                                                                                                                                               | 等级 |
|----|-----------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| S1 | ICLR 2025 Proceedings | MindSearch: Mimicking Human Minds Elicits Deep AI Searcher             | <a href="https://proceedings.iclr.cc/paper_files/paper/2025/hash/e01c431bbb83153632c0dcfaf8ccda0a-Abstract-Conference.html">https://proceedings.iclr.cc/paper_files/paper/2025/hash/e01c431bbb83153632c0dcfaf8ccda0a-Abstract-Conference.html</a> | A  |
| S2 | OpenReview            | MindSearch submission / paper page                                     | <a href="https://openreview.net/forum?id=xgtXkyqw1f">https://openreview.net/forum?id=xgtXkyqw1f</a>                                                                                                                                               | A  |
| S3 | arXiv                 | MindSearch: Mimicking Human Minds Elicits Deep AI Searcher, 2407.20183 | <a href="https://arxiv.org/abs/2407.20183">https://arxiv.org/abs/2407.20183</a>                                                                                                                                                                   | A  |
| S4 | GitHub official repo  | InternLM/MindSearch README, code tree, current repo status             | <a href="https://github.com/InternLM/MindSearch">https://github.com/InternLM/MindSearch</a>                                                                                                                                                       | A  |
| S5 | GitHub Releases       | MindSearch v0.1.0 release notes                                        | <a href="https://github.com/InternLM/MindSearch/releases">https://github.com/InternLM/MindSearch/releases</a>                                                                                                                                     | A  |
| S6 | GitHub Issues / PRs   | Current engineering activity and known integration issues              | <a href="https://github.com/InternLM/MindSearch/issues">https://github.com/InternLM/MindSearch/issues</a>                                                                                                                                         | B  |
| S7 | alphaXiv mirror       | Paper tables/appendix text indexing for ablations and time analysis    | <a href="https://www.alphaxiv.org/abs/2407.20183">https://www.alphaxiv.org/abs/2407.20183</a>                                                                                                                                                     | B  |

建议阅读顺序：先看论文 Figure 1/方法 → WebPlanner → WebSearcher → Table 1 → Planner/Searcher ablations → time efficiency & error correction appendix → 再读官方 README 了解可运行实现。

证据等级说明：A = 官方论文/官方仓库；B = 官方 issue 或高质量论文镜像，用于补充工程状态或便于定位，关键结论需与 A 级来源一致。

### 最终结论

MindSearch 是 Deep Research 发展史里非常关键的一步：它第一次把“搜索过程的思考结构”明确变成 Dynamic Graph，并用 Multi-Agent 将大规模网页读取从主规划上下文中隔离。它没有解决 2026 年 Deep Research 的全部问题，但它提出的 Graph State、层级检索、上下文分片和可验证规划仍然是非常值得保留的设计原语。